

NEETHI.AI

A project Report

Submitted to



**STATE BOARD OF TECHNICAL EDUCATION AND TRAINING,
ANDHRA PRADESH**

In the partial fulfilment of the requirements for the award of the degree of

DIPLOMA IN COMPUTER ENGINEERING

By

P. Kundhana Sree (23018-CM-018)

S. Vedha Chowdhary (23018-CM-020)

K. Harsha Vardhan (23018-CM-043)

C. Purushotham (23101-CM-006)

D. Deekshith (23155-CM-010)

Under The Esteemed Guidance of

G. Hari Sankar, M. Tech

Lecturer

Department of Computer Engineering



SRI VENKATESWARA GOVERNMENT POLYTECHNIC

**Affiliated to State Board of Technical Education and Training AP, Opp. T.T.D
Administration Building, K. T. Road, Tirupati-517501, Andhra Pradesh.**

Department of Computer Engineering



CERTIFICATE

This is to certify that the project entitled “NEETHI.AI” is
a bonafide work done

by

P. Kundhana Sree (23018-CM-018) S. Vedha Chowdhary (23018-CM-020)

K. Harsha Vardhan (23018-CM-043) C.Purushotham (23101-CM-006)

D. Deekshith (23155-CM-010)

*For the partial fulfilment of the requirements for the award of degree of Diploma in
Computer Engineering, from State Board of Technical Education and Training,
Andhra Pradesh during 2023-2026.*

Project Guide

G. Hari Sankar, M. Tech

Lecturer

Department of CME

SV Govt. Polytechnic.

Head of the Department

G. Prabakar, M. Tech

HCMES

Department of CME

SV Govt. Polytechnic.

Date: _____

Head Of the Department

Internal Examiner

External Examiner

DECLARATION

We hereby declare that the project report entitled “**NEETHI.AI**” done by us under the guidance of **G. HARI SANKAR**, Lecturer of Department of Computer Engineering and is submitted in partial fulfilment of requirements for the award of the degree of **Computer Engineering**. This report is the result of our effort and it has not been submitted to any other University or Institution for the award of any degree and diploma other than specified above.

Date:

Place:

P. Kundhana Sree (23018-CM-018)

S. Vedha Chowdhary (23018-CM-020)

K. Harsha Vardhan (23018-CM-043)

C. Purushotham (23101-CM-006)

D. Deekshith (23155-CM-010)

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude to our guide **G. Hari Sankar, Lecturer in Computer Engineering**, for his valuable guidance, encouragement, and constant support throughout our project. His helpful nature and insightful suggestions greatly contributed to the successful completion of our work.

We also express our sincere thanks to **G. Prabakar, M. Tech, Head of the Computer Engineering Section**, for his kind assistance, motivation, and encouragement during our course of study and the successful execution of this project.

Our deep sense of appreciation goes to our respected **Principal Dr. Y. Dwarakanath Reddy, M.Sc., Ph.D.**, for dedicating his valuable time and providing us with the opportunity and guidance to complete this project successfully.

Finally, we express our sincere gratitude to the **Management and our fellow students** for their continuous support, cooperation, and for providing the necessary facilities that made this project possible.

By

P. Kundhana Sree	(23018-CM-018)
S. Vedha Chowdhary	(23018-CM-020)
K. Harsha Vardhan	(23018-CM-043)
C. Purushotham	(23101-CM-006)
D. Deekshith	(23155-CM-010)

ABSTRACT

In today's digital era, the proliferation of misinformation and malicious content has become a critical challenge, undermining public trust, endangering personal security, and disrupting social harmony. The rapid dissemination of fake news and unsafe web links across social media, messaging platforms, and online news outlets makes it increasingly difficult for users to distinguish between authentic and deceptive information. To address this pressing issue, this project introduces a comprehensive, real-time Fake News and Malicious Content Detection Platform.

The platform leverages state-of-the-art artificial intelligence models and integrates with multiple trusted external APIs to cross-verify the authenticity of news articles and the safety of URLs. By analysing content against a wide array of reliable sources, the system provides users with accurate and timely verdicts on the credibility of information they encounter online. The solution features a user-friendly, mobile-responsive web interface and seamless WhatsApp integration, ensuring accessibility and convenience for a diverse user base. Multilingual support further broadens the platform's reach, enabling users from different linguistic backgrounds to benefit from its services.

Security and privacy are foundational to the platform's design, with robust encryption, secure data handling, and compliance with global standards such as GDPR and India-DPDP. The architecture is built for scalability and high performance, capable of handling large volumes of verification requests with minimal latency. By empowering users to make informed decisions and avoid digital threats, the platform aims to foster a safer, more trustworthy online environment and contribute to the global fight against misinformation and cybercrime.

CONTENTS

CHAPTER NO	DESCRIPTION	PAGE NO
1	Introduction	1
2	Project Description 2.1 Scope 2.2 Objective 2.3 Problem Statement 2.4 Why We Chose This Project	2 - 4
3	System Analysis 3.1 Existing system 3.2 Proposed systems	5 - 6
4	Computational environment 4.1 Software Requirements 4.2 Hardware Requirements 4.3 Non-Functional Requirements	7 - 8
5	Feasibility Study 5.1 Technical Feasibility 5.2 Operational Feasibility 5.3 Economical Feasibility	9

6	SYSTEM DESIGN 6.1 Architecture 6.2 UML Diagram	10 - 16
7	SYSTEM IMPLEMENTATION 7.1 MODULES	17
8	SAMPLE SOURCE CODE	18 - 32
9	TESTING	33 - 34
10	SCREEN LAYOUTS	35 - 37
11	FUTURE ENHANCEMENTS	38
12	CONCLUSION	39
13	REFERENCES	40

1. INTRODUCTION

In the modern digital era, the internet has revolutionized the way information is created, shared, and consumed. While this has brought about unprecedented access to knowledge and global connectivity, it has also given rise to significant challenges—most notably, the rapid spread of misinformation, fake news, deepfake images, and malicious URLs. These threats not only mislead individuals but can also cause widespread panic, financial loss, reputational damage, and even threaten democratic processes and public safety.

The proliferation of fake news and malicious content is exacerbated by the speed and reach of social media, messaging platforms, and online news outlets. Traditional fact-checking methods are often too slow and limited in scope to keep up with the volume and velocity of digital content. Moreover, the diversity of languages and regional dialects further complicates the detection of misinformation, leaving large populations vulnerable.

Recognizing these challenges, our project aims to develop a comprehensive, AI powered platform capable of detecting fake news, deepfake images, and unsafe URLs in real time. The platform leverages advanced machine learning models, real-time cross-verification with trusted sources, and multilingual support to provide users with instant, reliable, and actionable feedback. By integrating with popular communication channels such as WhatsApp and supporting both desktop and mobile environments, the platform ensures accessibility and ease of use for a broad audience.

Our mission is to empower individuals, organizations, and communities to make informed decisions, foster digital trust, and create a safer online environment. Through continuous innovation, ethical AI practices, and a commitment to user privacy and security, this platform aspires to set new standards in the fight against digital misinformation and cyber threats.

2. PROJECT DESCRIPTION

This project is a real-time fake news and malicious URL detection platform designed to help users quickly verify the authenticity of news articles and the safety of web links. Leveraging advanced AI models and trusted external APIs, the system crosschecks news content and URLs against multiple reliable sources. With a user-friendly web interface and WhatsApp integration, the platform provides instant, multilingual feedback, empowering individuals to make informed decisions and stay protected from misinformation and online threats.

2.1 Scope

The scope of this project encompasses the design, development, and deployment of a robust web application dedicated to detecting fake news, deepfake images, and malicious URLs in real time.

The platform is intended to serve a diverse user base, including individuals, organizations, and enterprises, by providing:

- **Intuitive User Interface:** A responsive, mobile-first design that ensures ease of use across devices and platforms.
- **Advanced AI Integration:** Utilization of state-of-the-art AI models for content analysis, crossverification, and summarization.
- **Multilingual Support:** Capability to process and analyse content in multiple languages and native scripts, making the platform accessible to users worldwide.
- **External API Integration:** Real-time connections to trusted news sources, URL scanning services, and messaging platforms for comprehensive verification.
- **Security and Privacy:** Implementation of end-to-end encryption, secure API key management, and compliance with GDPR and India-DPDP regulations.
- **Scalability:** Architecture designed for horizontal scaling, supporting high-volume usage and future expansion.
- **User Notifications:** Instant feedback and alerts delivered via web interface and WhatsApp integration.

The project also considers future enhancements such as mobile app development, blockchain integration for data integrity, and gamification to encourage user engagement.

2.2 Objective

The primary objectives of the platform are:

- **Accurate Detection:** To develop a system that can accurately identify and flag fake news, deepfake images, and malicious URLs using AI-powered analysis and real-time crossverification.
- **User Empowerment:** To provide users with instant, actionable feedback on the authenticity of content, enabling informed decision-making.
- **Accessibility:** To ensure the platform is accessible to users in multiple languages and on various devices, including integration with WhatsApp for widespread reach.
- **Security and Compliance:** To protect user data through robust security measures and ensure compliance with international privacy laws.
- **Scalability and Reliability:** To build a system capable of handling large volumes of verification requests with high uptime and reliability.
- **Continuous Improvement:** To maintain a modular, maintainable codebase that supports ongoing enhancements and adaptation to emerging threats.

2.3 Problem Statement

The digital landscape is plagued by the rapid spread of misinformation and malicious content. Key challenges include:

- **Speed of Spread:** Fake news and harmful links can go viral within minutes, outpacing traditional verification methods.
- **Language Barriers:** Most existing solutions are limited to major languages, leaving regional and local language users vulnerable.
- **Fragmented Tools:** Users often need to rely on multiple, unconnected tools for news, image, and URL verification, leading to inefficiency and confusion.
- **Lack of Real-Time Detection:** Many platforms cannot provide instant verification, reducing their effectiveness in fast-moving digital environments.

- **Security Risks:** Malicious URLs and phishing attacks continue to evolve, requiring constant vigilance and up-to-date detection mechanisms.
- **Trust Deficit:** The absence of a unified, reliable platform erodes user trust and increases susceptibility to misinformation.
- This project aims to address these issues by providing a unified, real-time, multilingual platform for comprehensive content verification.

2.4 Why We Chose This Project

The decision to undertake this project is driven by the urgent need to combat the global crisis of misinformation and digital threats. Key motivations include:

- **Societal Impact:** Misinformation and cyber threats have far-reaching consequences, affecting public health, safety, and democracy.
 - **Technological Gap:** There is a lack of comprehensive, real-time, and multilingual solutions in the current market.
 - **Ethical Responsibility:** As technology developers, we have a duty to create tools that promote truth, trust, and digital safety.
 - **Innovation Opportunity:** The integration of advanced AI, real-time data, and user centric design presents a unique opportunity to set new standards in digital verification. •
- Accessibility and Inclusion:** By supporting multiple languages and platforms, we aim to make digital safety tools accessible to all, including underserved communities.
- **Mission-Driven Approach:** Inspired by the principle of ethical intelligence, our project aspires to make a positive difference in the digital world.

3. SYSTEM ANALYSIS

System analysis involves understanding the limitations of existing solutions and defining how the proposed platform addresses these gaps.

3.1 Existing System

Current solutions for fake news and malicious content detection are often fragmented and lack real-time capabilities, comprehensive language support, or unified user experiences. Users must rely on multiple tools for different types of verification, leading to inefficiency, confusion, and gaps in protection. Many platforms do not provide instant feedback or support for regional languages, leaving significant portions of the population vulnerable to misinformation and cyber threats.

3.2 Proposed System

The proposed platform offers a unified, AI-driven solution for real-time verification of news, images, and URLs. Key features include:

- **Multilingual Support:** The system can process and analyze content in global and regional languages, including native scripts, making it accessible to a diverse user base.
- **Comprehensive Verification:** Integration with trusted news sources and security APIs enables robust cross-verification of content.
- **User-Friendly Interfaces:** Both web and WhatsApp interfaces are provided for easy access and instant feedback.
- **Secure Data Handling:** Encryption, secure API keys, and compliance with privacy laws ensure user data is protected.
- **Scalable Architecture:** The system is designed to handle high volumes of verification requests and can be easily expanded to support new features.

Advantages

1. **Efficiency:** Real-time detection and instant feedback streamline the verification process for users.
2. **Reliability:** AI-powered cross-verification and trusted data sources ensure high accuracy and trustworthiness.
3. **User-Friendly Interface:** The intuitive design caters to both desktop and mobile users, reducing barriers to adoption.

4. Scalability: The architecture supports growth in user demand and the addition of new features.
5. Security: End-to-end encryption and robust access controls protect user data and system integrity.
6. Community Trust: Transparent privacy policies and ethical AI practices foster user confidence and engagement.

Disadvantages

1. Dependence on External APIs: The platform's reliance on third-party APIs for verification and scanning means any downtime or changes in these services can disrupt functionality.
2. False Positives/Negatives: Even with advanced AI, the system may occasionally misclassify content, either flagging real information as fake or missing sophisticated misinformation.
3. Language and Context Challenges: Accurately detecting fake content in diverse languages and regional dialects remains difficult, especially with evolving slang and cultural nuances.
4. Privacy Concerns: Handling user-submitted content, especially via messaging platforms, raises privacy and data protection issues that may deter some users.
5. Resource Intensive: Real-time AI processing and verification require significant computational resources, potentially increasing operational costs and limiting scalability.
6. Evolving Threats: Misinformation tactics and cyber threats are constantly changing, requiring ongoing updates and improvements to maintain platform effectiveness.

4. COMPUTATIONAL ENVIRONMENT

A robust computational environment is essential for the successful development, deployment, and operation of the Fake News and Malicious Content Detection Platform. This chapter outlines the software and hardware requirements, as well as the non-functional attributes that ensure the system's performance, security, and scalability.

4.1 Software Requirements

- **Operating System:** The platform is compatible with Windows 10+ and macOS for both development and deployment, ensuring flexibility and broad accessibility.
- **Web Browsers:** The user interface is optimized for Chrome (95+), Safari (14+), Firefox, and Edge, providing a seamless experience across devices.
- **Server Environment:** Node.js is used for backend development, offering a scalable and efficient runtime for handling API requests and real-time processing. XAMPP is utilized for local development and testing.
- **Database:** MySQL serves as the primary database, providing robust, secure, and scalable data storage for user information, verification logs, and system configurations.
- **Frontend Technologies:** HTML, CSS, and JavaScript are employed to create a responsive, mobile-first user interface.
- **Automation/Workflow:** n8n is integrated for orchestrating background tasks, API calls, and workflow automation.
- **External APIs:** The platform connects to advanced AI models (Gemini, OpenRouter, Perplexity) for language processing and fact-checking, as well as security APIs (Tavily, Google Safe Browsing, PhishTank, VirusTotal) for URL scanning. Ultrams is used for WhatsApp integration.
- **Development Tools:** Visual Studio Code and Notepad++ are recommended for code editing and debugging.

4.2 Hardware Requirements

- **Development Machine:** A computer or laptop with at least 8GB RAM and 512GB hard disk is required for smooth development and testing.
- **Input Devices:** Standard mouse and keyboard for user interaction.
- **Network:** A stable internet connection with HTTPS/TLS 1.2+ support is essential for secure communication and real-time API integration.
- **Hosting:** The platform is deployed on a Render cloud server or equivalent, ensuring high availability and scalability.

4.3 Non-Functional Requirements

- **Performance:** The system is optimized for API responses under 2 seconds for typical verification requests, ensuring a smooth user experience.
- **Scalability:** The architecture supports horizontal scaling via containers and autoscaling groups, accommodating growth in user base and verification volume.
- **Security:** End-to-end HTTPS encryption, encrypted data at rest, secure API key management, and role-based access controls protect user data and system integrity.
- **Reliability:** The platform guarantees 99.9% uptime, with automatic failover and regular backups to prevent data loss and service interruptions.
- **Usability:** The user interface is designed to be intuitive and mobile-first, with clear feedback and guidance throughout the verification process.
- **Maintainability:** A modular codebase, comprehensive documentation, and a CI/CD pipeline facilitate easy updates and ongoing enhancements.
- **Privacy:** The platform is fully compliant with GDPR and India-DPDP regulations, ensuring minimal data retention and a transparent privacy policy.

5. FEASIBILITY STUDY

A feasibility study evaluates the practicality and viability of the proposed platform from technical, operational, and economic perspectives.

5.1 Technical Feasibility

The project leverages mature, widely adopted technologies such as Node.js, MySQL, and modern frontend frameworks, ensuring a stable and scalable foundation. Integration with leading AI models and security APIs is facilitated through RESTful interfaces, making realtime verification achievable. The modular architecture allows for easy addition of new features and adaptation to emerging threats. Rigorous testing, continuous integration, and automated deployment practices further enhance the platform's technical robustness.

5.2 Operational Feasibility

Operationally, the platform is designed for ease of use and efficient management. The intuitive, mobile-first interface reduces the learning curve for users, while automated workflows and real-time notifications streamline operations for both users and administrators. The system's adaptability allows for quick responses to new types of misinformation and user needs, ensuring long-term sustainability and relevance.

5.3 Economic Feasibility

The economic feasibility of the project is strong. Cloud-based deployment minimizes infrastructure costs, while automation reduces the need for manual intervention, lowering operational expenses. The platform's potential for a large user base, enterprise partnerships, and premium features offers significant revenue opportunities. Compliance with privacy and security standards reduces legal risks, supporting sustainable growth and long-term viability.

6. SYSTEM DESIGN

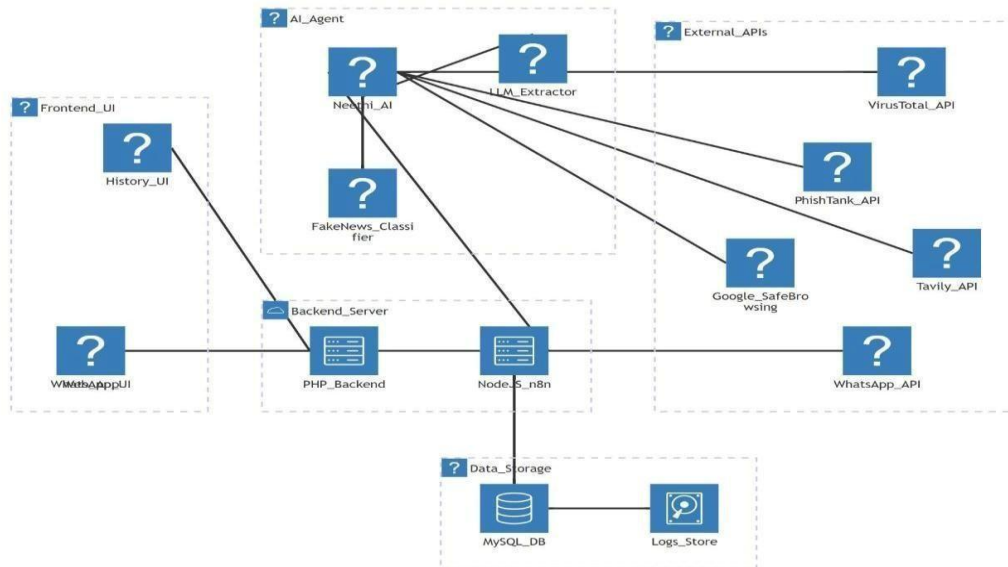
6.1 Architecture:

This diagram illustrates the overall architecture of a fake news detection and alerting system. The main components are:

- **Frontend UI:** Includes interfaces like History UI and WhatsApp UI for user interaction.
- **Backend Server:** Consists of PHP Backend and Node.js, which handle business logic and workflow automation.
- **AI Agent:** Contains modules like Neethi AI, LLM Extractor, and Fake News Classifier for content analysis.
- **External APIs:** Integrates with third-party services such as Virus Total API, Phish Tank API, Tavily API, Google Safe Browsing, and WhatsApp API for threat intelligence and messaging.
- **Data_Storage:** Uses MySQL DB and Logs Store for storing verification records and logs.

Data Flow:

- Users interact with the Frontend UI, which communicates with the Backend Server.
- The Backend Server processes requests and interacts with AI Agent modules for content analysis.
- AI Agent modules may call External APIs for additional verification.
- Results and logs are stored in Data Storage.
- Notifications and alerts are sent via WhatsApp API.



6.2 UML Diagram:

Unified Modelling Language (UML) is a standard way to visually describe and design software systems, as well as business processes and other complex systems. It's like a common language that helps teams plan, build, and document how everything in a system should work.

UML is based on proven engineering practices that make it easier to model large and complicated systems. It's especially important for object-oriented software development, where you build systems out of reusable pieces called objects.

Most UML diagrams use simple graphics and symbols to show how different parts of a project fit together. This visual approach helps teams communicate ideas clearly, check if the design makes sense, and make sure everyone is on the same page.

System design is all about figuring out what parts a system needs (like components, modules, and data), how they connect, and how they work together to meet the project's goals. UML diagrams make this process much easier.

What's in a UML Diagram?

A typical UML diagram includes:

- **Entities:** These are the main parts of the system, such as classes, objects, users, or behaviours.
- **Relationship lines:** These lines show how entities are connected or interact with each other.

Types of Relationships:

- **Generalization:** A solid line with an arrow pointing to a more general or abstract entity (like a parent class).
- **Association:** A solid line showing that one entity uses or interacts with another.
- **Dependency:** A dotted line with an arrow, showing that one entity depends on another's behaviour.

UML Diagrams in This Project

In this project, four basic UML diagrams are explained:

1. Use Case Diagram

2. Sequence Diagram

These diagrams help visualize how the system works and how different parts interact, making the design process clearer and more organized.

1. Use Case Diagram

Use Case Diagram Explanation

This diagram represents the main interactions between users, the system, and administrators in a fake news detection and alerting platform.

Actors

1. **User** ○ The primary actor who interacts with the system to verify news, view history, and receive alerts.
2. **Admin** ○ The administrator who manages users and monitors fake news trends.

User Use Cases

- **Submit News/URL/Image**
 - The user can submit news articles, URLs, or images to the system for verification.
- **Analyze Content** ○ The system analyzes the submitted content using AI and other tools to determine its authenticity.
- **Get Real/Fake Result** ○ After analysis, the user receives a result indicating whether the content is real or fake.

- **View History** ○ The user can view the history of their previous submissions and results.
- **Receive Real-Time Alerts** ○ The user receives notifications if fake news is detected, especially if it is relevant to their region or interests.

AI Agent Use Cases

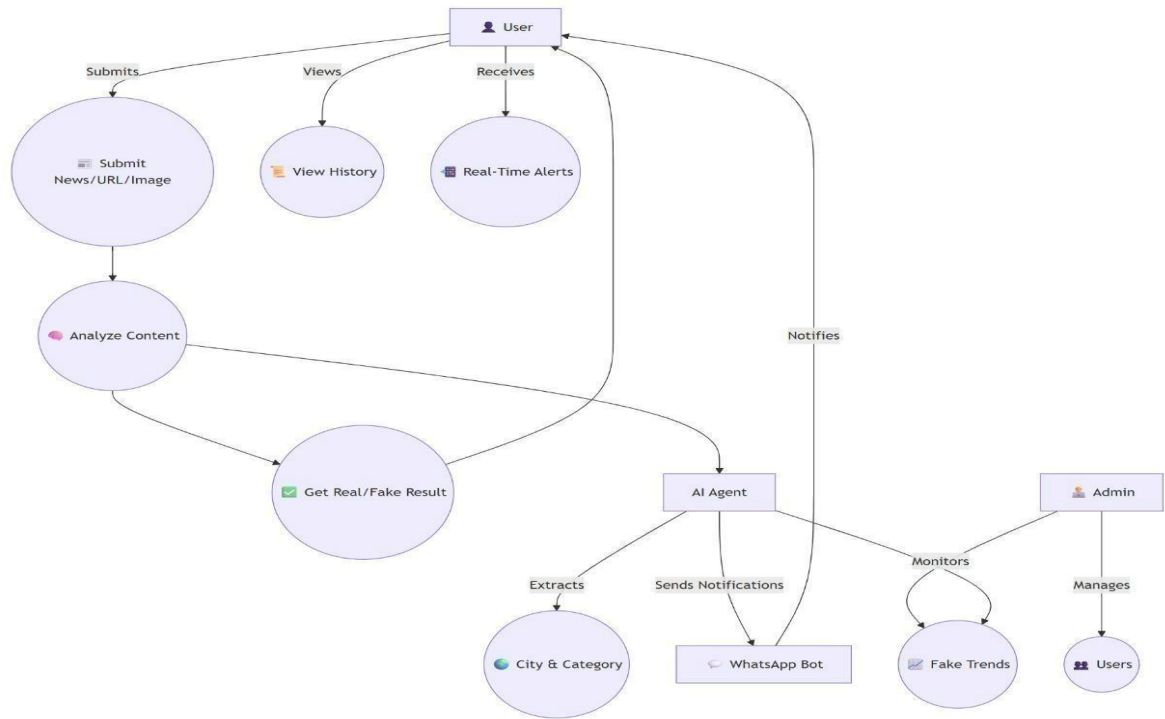
- **Extracts City & Category** ○ The AI agent extracts location and category information from the analyzed content for targeted alerts.
- **Sends Notifications (WhatsApp Bot)** ○ The AI agent sends notifications to users, often via a WhatsApp bot, to inform them about fake news or verification results.
- **Monitors Fake Trends** ○ The AI agent tracks and analyzes trends in fake news to identify patterns and emerging threats.

Admin Use Cases

- **Monitors Fake Trends** ○ The admin can view analytics and reports about fake news trends in the system.
- **Manages Users** ○ The admin manages user accounts, permissions, and possibly handles escalations or misuse.

Flow Summary

1. **User submits content** for verification.
2. The system **analyzes the content** and determines if it is real or fake.
3. The **result is provided** to the user.
4. The **AI agent** may extract additional information (like city and category) and **send notifications** via WhatsApp.
5. The user can **view their history** and **receive real-time alerts**.
6. The **admin monitors trends** and **manages users**.



1. Sequence Diagram:

A **sequence diagram** is a type of drawing used in software design to show how different parts of a system interact with each other over time. Think of it as a stepby-step story of how things happen in a system.

It shows:

- Which parts (or objects) are involved,
- What actions or messages they send to each other,
- And the exact order in which these actions happen.

This makes sequence diagrams a great way to visualize and understand how a system actually works when it's running, especially when you want to see the flow of events or how different pieces work together to get something done.

Main Flow: From Submission to Result

1. **User submits news/article via the frontend (web or WhatsApp):** ◦ The process begins when a user encounters a suspicious news article, URL, or image and wants to verify its authenticity. ◦ The user accesses the system through a web interface or WhatsApp and submits the content for verification.

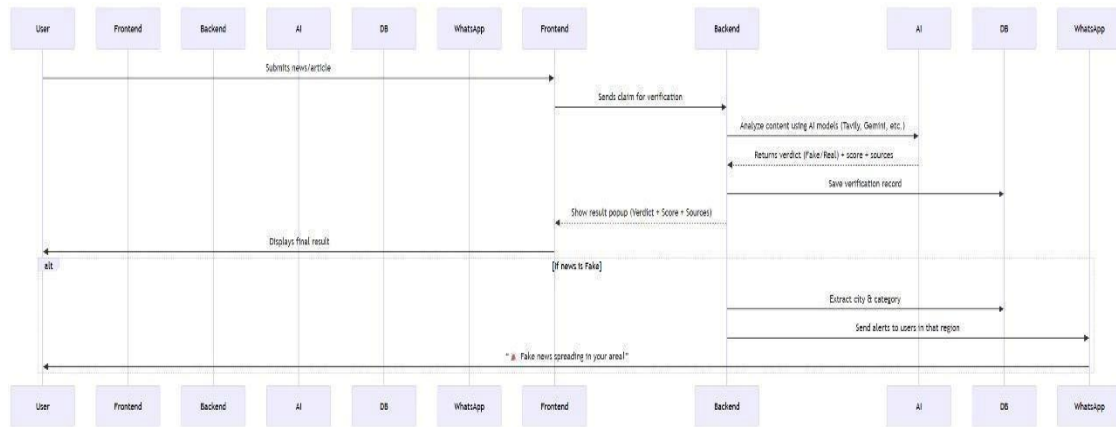
2. **Frontend sends the claim to the backend for verification:** ◦ The frontend application collects the user's submission and forwards it to the backend server. ◦ This step ensures the user's input is securely and reliably transmitted for processing.
3. **Backend analyzes the content using AI models (like Tavily, Gemini, etc.):** ◦ The backend receives the submission and initiates the verification process. ◦ It leverages advanced AI models (such as Tavily, Gemini, or others) to analyze the content. ◦ The analysis may include checking for known fake news patterns, crossreferencing with trusted sources, and evaluating the credibility of the information.
4. **AI returns a verdict (Fake/Real), a confidence score, and sources:**
 - The AI models process the content and generate a result:
 - **Verdict:** Whether the content is likely fake or real.
 - **Confidence Score:** How certain the AI is about its verdict (e.g., 95% sure it's fake).
 - **Sources:** References or evidence supporting the verdict (e.g., links to fact-checking sites).
5. **Backend saves the verification record in the database:** ◦ The backend stores the result, along with the original submission and analysis details, in a database. ◦ This allows for future reference, auditing, and user history tracking.
6. **Frontend displays the result to the user (verdict, score, sources):**
 - The frontend retrieves the result from the backend and presents it to the user. ◦ The user sees whether the news is real or fake, the confidence score, and supporting sources, helping them make an informed decision.

Alternative Flow: If News is Fake

If the AI determines the news is fake, additional steps are triggered to protect the community:

1. **Backend extracts city and category information from the content:** The backend analyzes the content further to identify the geographical location (city/region) and the category (e.g., health, politics) relevant to the fake news.
2. **AI identifies the affected region:** The AI pinpoints which users or groups might be impacted by this fake news, based on their location or interests.

3. **Backend sends alerts to users in that region via WhatsApp:** The system automatically sends real-time alerts to users in the affected area using WhatsApp. This ensures that people who are most at risk of encountering the fake news are promptly warned.
4. **Frontend displays a warning:** Users accessing the system (web or WhatsApp) in the affected region see a prominent warning message.
5. **Warning message example:** “Fake news spreading in your area!” ,This message helps raise awareness and prevent the spread of misinformation.



7. SYSTEM IMPLEMENTATION

System implementation details the practical realization of the platform's design, including the development of core modules and integration of key technologies.

7.1 Modules

7.1.1 User Module

- **Functionality:** Allows users to submit news articles, images, or URLs for verification.
- **Features:** Provides instant feedback, detailed reports, and a history of previous checks. Supports multiple languages and formats.

7.1.2 Admin Module

- **Functionality:** Enables administrators to manage user accounts, monitor system health, and review flagged content.
- **Features:** Includes dashboards for analytics, compliance monitoring, and security oversight.

7.1.3 Integration Module

- **Functionality:** Manages connections to external APIs for news verification, URL scanning, and messaging.
- **Features:** Ensures seamless, real-time data exchange and error handling.

7.1.4 Notification Module

- **Functionality:** Sends alerts and verification results to users via WhatsApp and web notifications.
- **Features:** Supports customizable notification preferences and real-time updates.

7.1.5 Security Module

- **Functionality:** Handles authentication, authorization, and encryption.
- **Features:** Implements role-based access control, secure API key management, and compliance checks.

Implementation Approach

The platform is developed using agile methodologies, with iterative development cycles, regular testing, and continuous integration. Each module is built as an independent, reusable component, facilitating easy maintenance and future enhancements.

8. SAMPLE SOURCE CODE

Logic code for Checking News:

```
// Signup form validation and submission
document.getElementById("signupSubmit").addEventListener("click", async () =>
{   const name = document.getElementById("signupName").value;   const mobile
= document.getElementById("signupMobile").value;   const pin =
document.getElementById("signupPin").value;

    let isValid = true;

    // Clear previous validation messages   hideValidationMessage("signupNameError");
hideValidationMessage("signupMobileError");
hideValidationMessage("signupPinError");

    // Validate name   if (name.trim() === "" || !validateName(name)) {
showValidationMessage("signupNameError", "Please enter a valid name");
        isValid = false;
    }

    // Validate mobile   if (mobile.trim() === "" ||
!validateMobile(mobile)) {       showValidationMessage("signupMobileError", "Please
enter a valid 10-digit mobile number");
        isValid = false;
    }

    // Validate PIN   if (pin.trim() === "" ||
!validatePin(pin)) {
showValidationMessage("signupPinError",
```

```

"Please enter a 4-digit PIN");

isValid = false;

}

if (isValid) {
    isValid = await createAccount(name, mobile, pin);
}

if (isValid === true) {
    userFlowType = 'signup';
    hideSignupPopup();
    // For signup, show questions popup
    showQuestionsPopup();
}

else if (isValid.message === "System Failure! Try again.") {
    showValidationMessage("signupPinError", "System Failure! Try again.");
}

else {
    showValidationMessage("signupPinError", "Already you have created Account! Login
now.");
}

});

// Login form validation and submission
document.getElementById("loginSubmit").addEventListener("click", async () =>
{
    const mobile = document.getElementById("loginMobile").value;
    const pin =
document.getElementById("loginPin").value;

    let isValid = true;

```

```

// Clear previous validation messages hideValidationMessage("loginMobileError");
hideValidationMessage("loginPinError");

// Validate mobile  if (mobile.trim() === "" ||
!validateMobile(mobile)) {      showValidationMessage("loginMobileError", "Please
enter a valid 10-digit mobile number");
    isValid = false;
}

// Validate PIN  if (pin.trim() === "" || !validatePin(pin)) {
showValidationMessage("loginPinError", "Please enter a 4-digit PIN");
    isValid = false;
}

if (isValid) {
    isValid = await loginMyAccount(mobile, pin);
    if (isValid.success) {
        const newsInput = document.getElementById("newsInput").value;
        userFlowType = 'login';
hideLoginPopup();
        // For login, go directly to final processing
showFinalProcessingPopup();

        // Show loading animation and wait 3 seconds at 90%
document.getElementById("processingTitle").textContent = "Almost Done!";
document.getElementById("finalMessage").textContent = "Analyzing your news...";

        checkNews(newsInput);
    }
}

```

```

else {      showValidationMessage("loginPinError",
isValid.message);
    }
}
});

```

```

// Questions form validation and submission (only for signup)
document.getElementById("questionsSubmit").addEventListener("click", async () => {
const category = document.getElementById("categorySelect").value;    const source =
document.getElementById("sourceInput").value;

```

```

    let isValid = true;

```

```

    // Clear previous validation messages
hideValidationMessage("categoryError");    hideValidationMessage("sourceError");

```

```

    // Validate category selection    if (category === "") {
showValidationMessage("categoryError", "Please select a category");
    isValid = false;
}

```

```

    // Validate source input    if (source.trim() === "") {
showValidationMessage("sourceError", "Please enter your preferred news source");
    isValid = false;
}

```

```

    isValid = await sendQuestions(category, source);

```

```

    if (isValid.success) {
hideQuestionsPopup();    // Show

```

```

final processing popup
showFinalProcessingPopup();

    // Show loading animation and wait 3 seconds at 90%
document.getElementById("processingTitle").textContent = "Almost Done!";
document.getElementById("finalMessage").textContent = "Setting up your preferences and
analyzing news...";

    const newsInput = document.getElementById("newsInput").value;
checkNews(newsInput);
    }
});

// View Results button handler
document.getElementById("viewResults").addEventListener("click", async () => {
hideFinalProcessingPopup();
    // Reset state after completion
});

async function checkToken() {    const token =
localStorage.getItem('neethiToken') || null;    if (token) {
const request = await fetch('http://localhost:3000/check-token', {
method: 'POST',        headers: {
            'Authorization': token
        }
    });
    const response = await request.json();
if (response.valid) {        return true;
    }        else {
return false;

```

```

    } } else
{
return false;
}
}

```

```

async function createAccount(name, mobile, pin) {  const
request = await fetch('http://localhost:3000/signup', {    method:
'POST',    headers: {
    'Content-type': 'application/json'
  },
  body: JSON.stringify({ name, mobile, pin })
});
const response = await request.json();  if
(response.success) {    localStorage.setItem('neethiToken',
response.token);
  return true;
}
else if (response.message === 'System Failure!') {
  return { message: "System Failure! Try again." };
}
else {
return false;
}
}

```

```

async function loginMyAccount(mobile, pin) {
  const request = await fetch('http://localhost:3000/login',
{    method: 'POST',    headers: {
    'Content-type': 'application/json'

```

```

    },
    body: JSON.stringify({ mobile, pin })
  });

  const response = await request.json();  if
(response.success) {    localStorage.setItem('neethiToken',
response.token);
  }
  return response;
}

async function checkNews(news) {  const articleId =
Math.round((Math.random()) * 1000000);  const token =
localStorage.getItem('neethiToken');  const request = await
fetch('http://localhost:3000/check-news', {    method: 'POST',    headers:
{
      'Content-type': 'application/json'
    },
    body: JSON.stringify({ articleId, token, news })
  });

  const response = await request.json();  console.log(response);  if
(!response.code) {    updateProgressBar("finalProgressBar",
"finalProgressText", 100);

    // Change to success icon and update content    const loadingIcon
= document.querySelector(".loading-icon");    loadingIcon.className
= "success-icon";    loadingIcon.innerHTML = '<i class="fas fa-
check-circle"></i>';

```

```

    document.getElementById("processingTitle").textContent = "Analysis Complete!";
    document.getElementById("finalMessage").textContent = "Your news analysis is ready.";
    document.getElementById("viewResults").style.display = "block";

```

```

    // Store the result data for display    window.resultData
    = response;

```

```

    // Update the view results button to show the result popup
    document.getElementById("viewResults").onclick = function () {
        hideFinalProcessingPopup();    displayResult(response);
    };
}
else if (response.code === 69) {    hideFinalProcessingPopup();
showNotNewsPopup();
}
else { confirm('Workflow error');
    setTimeout(() =>
    {    location.reload();
    }, 2000);
}
}

```

```

async function sendQuestions(category, city) {    const token =
localStorage.getItem('neethiToken');    const request = await
fetch('http://localhost:3000/user-details', {    method: 'POST',    headers:
{
        'Content-type': 'application/json'
    },
    body: JSON.stringify({ token, category, city })
});

```



```

    const response = await request.json();    return
response;
}

const express = require('express'); const router
= express.Router(); const jwt =
require('jsonwebtoken'); const {
JWT_SECRET } = require('../config');

router.post('/check-news', async (req, res) => {
    try {
        const token = req.body.token;    const
news = req.body.news;    const articleId =
req.body.articleId;    if
(!token) res.json({ message: 'No token'
    })

        const decoded = jwt.verify(token, JWT_SECRET);    const
mobile = decoded.mobile;

        const request = await fetch('http://localhost:5678/webhook/84f8c6a0-c93e-43d6-
b816c6596b71a3f4', {
            method: 'POST',
headers: {
            'Content-type': 'application/json'
        },
        body: JSON.stringify({ articleId, mobile, news })
    });

    const response = await request.json();    res.json(response);

```

```

    } catch (error) {
console.log('Webhook error');
    }
});

```

```

module.exports = router;

```

Logic code for checking URL:

```

// View Results button handler

```

```

document.getElementById("viewResults").addEventListener("click", async () =>
{   hideFinalProcessingPopup();   displayResult();
});

```

```

// Clipboard paste functionality const urlInput = document.getElementById('urlInput');
const pasteButton = document.getElementById('pasteButton');

```

```

if (urlInput && pasteButton) {   urlInput.addEventListener('focus',
() => {       pasteButton.style.display = 'inline-block';
    });

```

```

    urlInput.addEventListener('blur', () => {
// small delay to allow clicking paste button
setTimeout(() => {       if
(!pasteButton.matches(':hover')) {           pasteButton.style.display
= 'none';
        }
    }, 100);
});

```

```

pasteButton.addEventListener('click', async () => {

```

```

    try {
        const text = await navigator.clipboard.readText();        urlInput.value =
text.trim();        const validationMessage =
document.getElementById('validationMessage');        if (validationMessage)
validationMessage.style.display = 'none';
    } catch (err) {        console.error('Failed to read clipboard contents: ', err);
const validationMessage = document.getElementById('validationMessage');
if (validationMessage) {        validationMessage.textContent = 'Failed to
paste. Please paste manually.';        validationMessage.style.display = 'block';

    }
}
});
}

```

```

async function checkToken() {    const token =
localStorage.getItem('neethiToken') || null;    if (token) {
const request = await fetch('http://localhost:3000/check-token', {
method: 'POST',        headers: {

            'Authorization': token
        }
    });
    const response = await request.json();
    if (response.valid) {        return true;
    }        else {
return false;
    }    }    else
{
return false;
    }
}

```

```
}
```

```
async function createAccount(name, mobile, pin) {  
  const request = await fetch('http://localhost:3000/signup',  
    {    method: 'POST',    headers: {  
      'Content-type': 'application/json'  
    },  
    body: JSON.stringify({ name, mobile, pin })  
  });  
  const response = await request.json();  if  
(response.success) {    localStorage.setItem('neethiToken',  
response.token);  
    return true;  
  }  
  else if (response.message === 'System Failure!') {    return  
{ message: "System Failure! Try again." };  
  }  else {  
return false;  
  }  
}
```

```
async function loginMyAccount(mobile, pin) {  
  const request = await fetch('http://localhost:3000/login',  
    {    method: 'POST',    headers: {  
      'Content-type': 'application/json'  
    },  
    body: JSON.stringify({ mobile, pin })  
  });  
  const response = await request.json();  if  
(response.success) {
```

```

localStorage.setItem('neethiToken',
response.token);

}
return response;
}

```

```

async function sendQuestions(category, city) {  const token =
localStorage.getItem('neethiToken');  const request = await
fetch('http://localhost:3000/user-details', {    method: 'POST',    headers:
{
      'Content-type': 'application/json'
    },
    body: JSON.stringify({ token, category, city })
  });
  const response = await request.json();  return
response;
}

```

```

async function checkUrl(url) {  const urlId =
Math.round((Math.random()) * 1000000);  const token =
localStorage.getItem('neethiToken');  const request = await
fetch('http://localhost:3000/check-url', {    method: 'POST',    headers:
{
      'Content-type': 'application/json'
    },
    body: JSON.stringify({ urlId, token, url })
  });
  const response = await request.json();
console.log(response);  if (!response.code) {

```

```

updateProgressBar("finalProgressBar", "finalProgressText", 100);

    // Change to success icon and update content    const
loadingIcon = document.querySelector(".loading-icon");
loadingIcon.className = "success-icon";    loadingIcon.innerHTML
= '<i class="fas fa-check-circle"></i>';

    document.getElementById("processingTitle").textContent = "Analysis Complete!";
document.getElementById("finalMessage").textContent = "Your Link analysis is ready.";
document.getElementById("viewResults").style.display = "block";

    // Store the result data for display
window.resultData = response;

    // Update the view results button to show the result popup
document.getElementById("viewResults").onclick = function () {
hideFinalProcessingPopup();    displayResult(response);
    };
}
else if (response.code == 69) {    hideFinalProcessingPopup();
showNotNewsPopup();
}
else {    confirm('n8n Workflow
error');    setTimeout(() => {
location.reload();
    }, 2000);
}
}

```

```

const express = require('express'); const router
= express.Router(); const jwt =
require('jsonwebtoken'); const {
JWT_SECRET } = require('../config');

router.post('/check-url', async (req, res) => {
  try {
    const token = req.body.token;
    const url = req.body.url;      const
    urlId = req.body.urlId;

    if (!token) res.json({ message: 'No token' })

    const decoded = jwt.verify(token, JWT_SECRET);      const
    mobile = decoded.mobile;

    const request = await fetch('http://localhost:5678/webhook/776e35c1-2e51-431e-bca1-
36960f702fce', {
      method: 'POST',
      headers: {
        'Content-type': 'application/json'
      },
      body: JSON.stringify({ urlId, mobile, url })
    });

    const response = await request.json();
    res.json(response);
  } catch (error) {
    console.log('Webhook error');
  }
});

```

9. TESTING

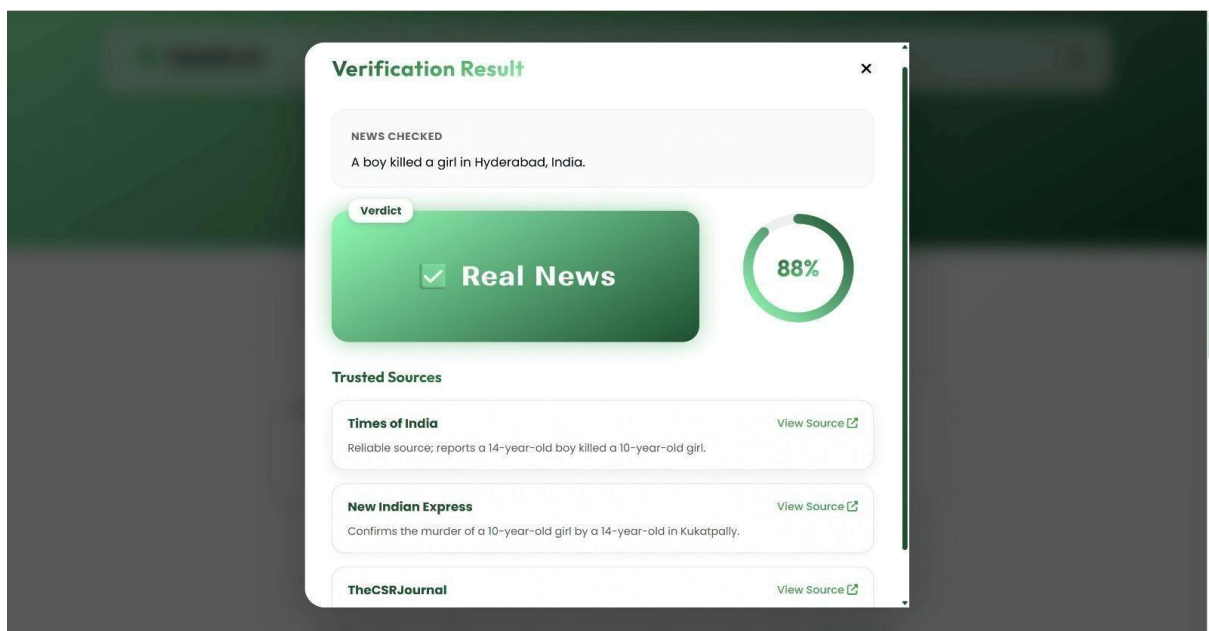
Testing is a critical phase in the software development lifecycle, ensuring that the platform is robust, secure, and delivers a seamless experience to users. A comprehensive testing strategy is employed to validate every aspect of the Fake News and Malicious Content Detection Platform, from individual components to the system as a whole.

Case Study 1: News Verification

Input: A boy killed a girl in Hyderabad, India

Expected Output: Real News

Output:



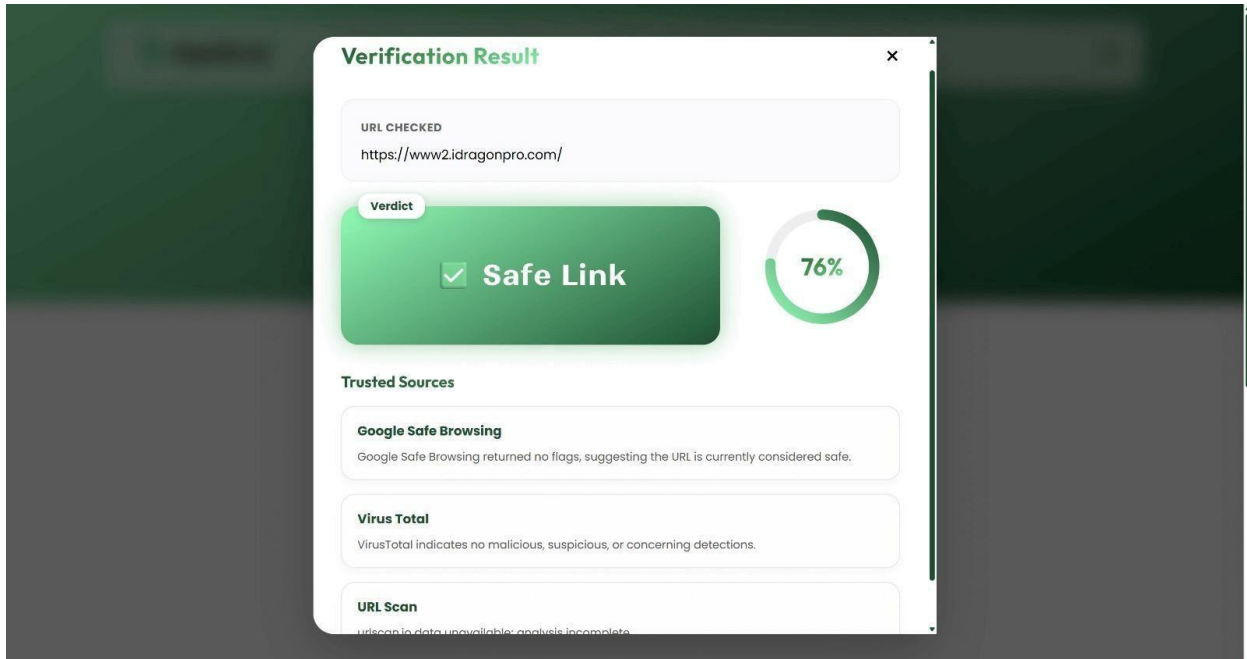
Case Study 2: Link Verification

Input:

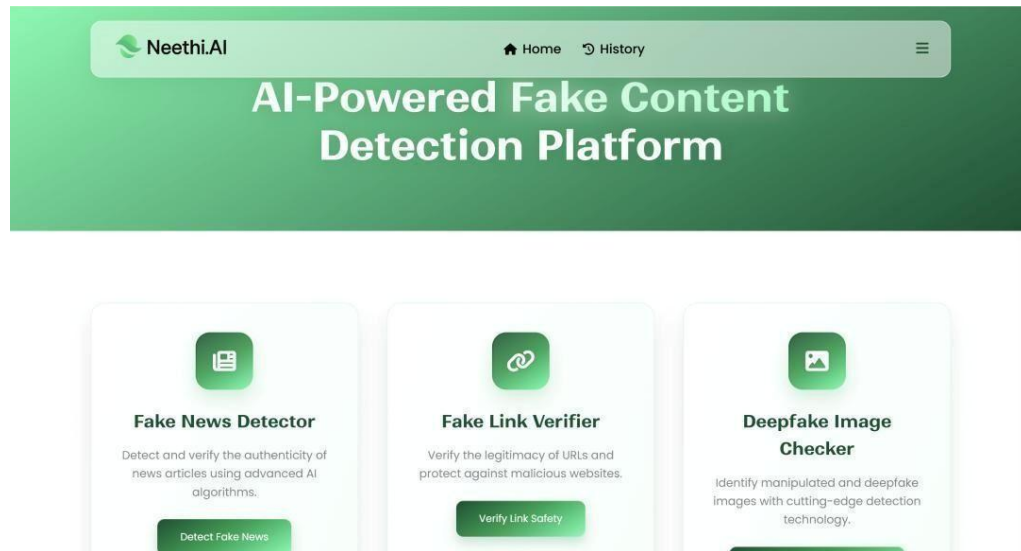
<https://www2.idragonpro.com/>

Expected Output: Safe Link

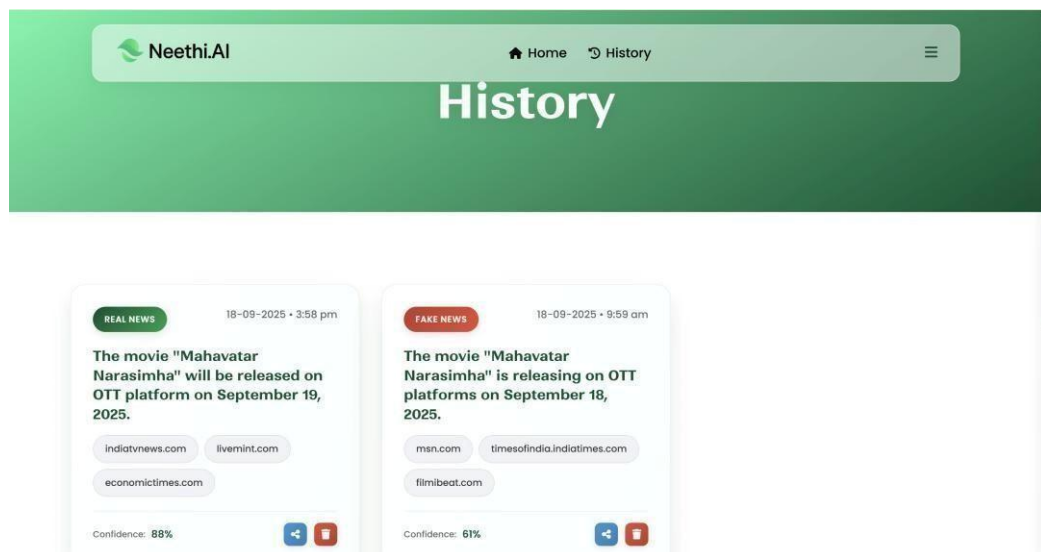
Output:



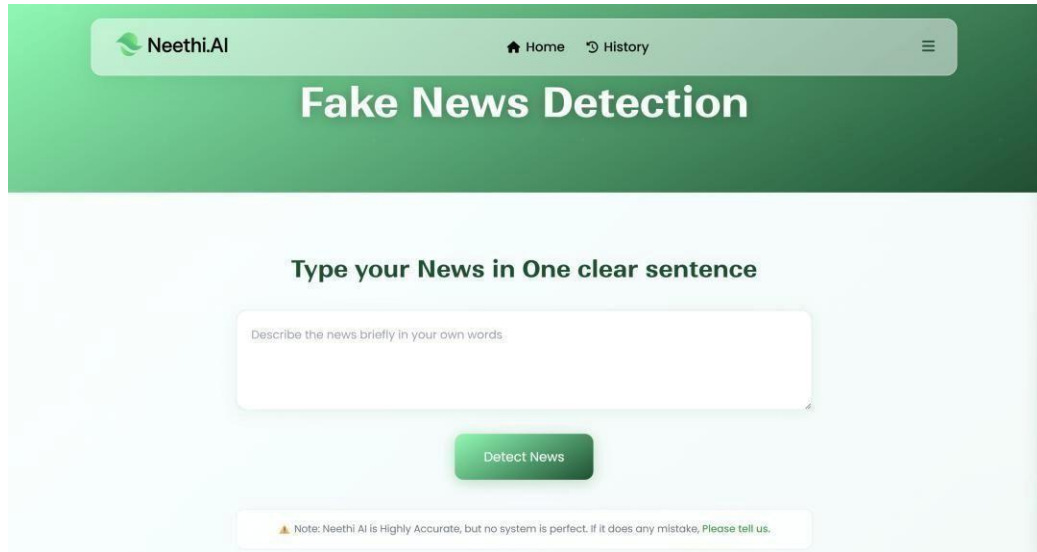
10. SCREEN LAYOUTS



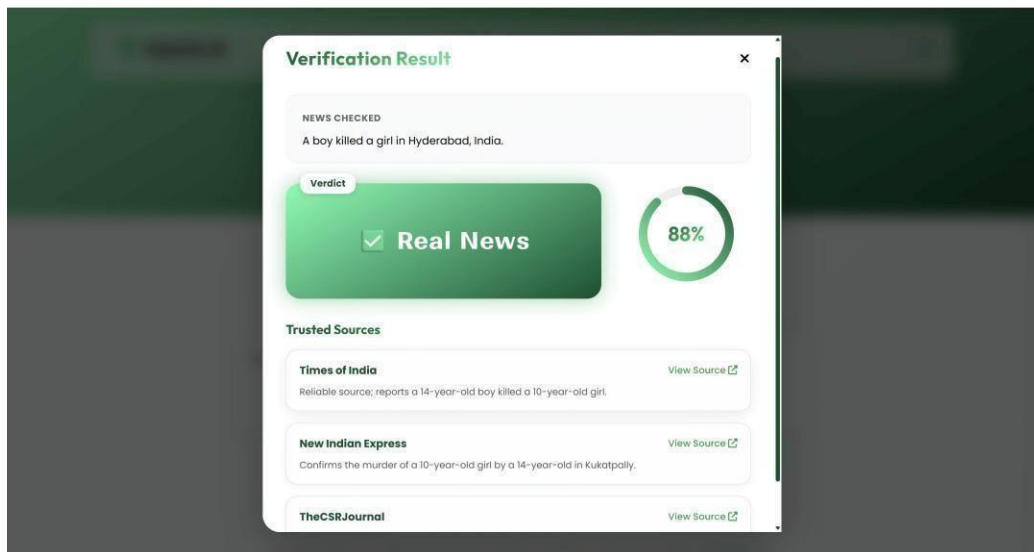
10.1. Home Page



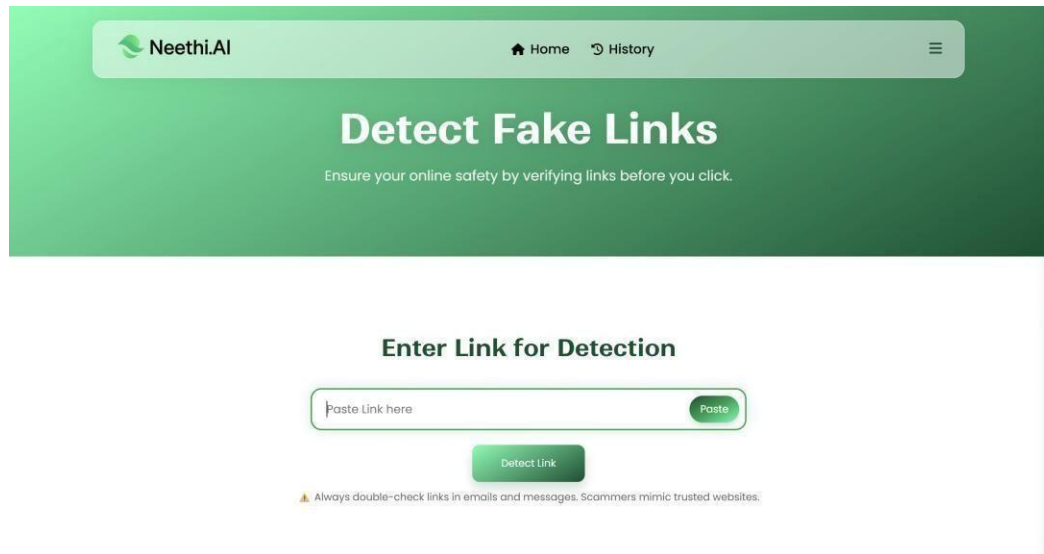
10.2. History Page



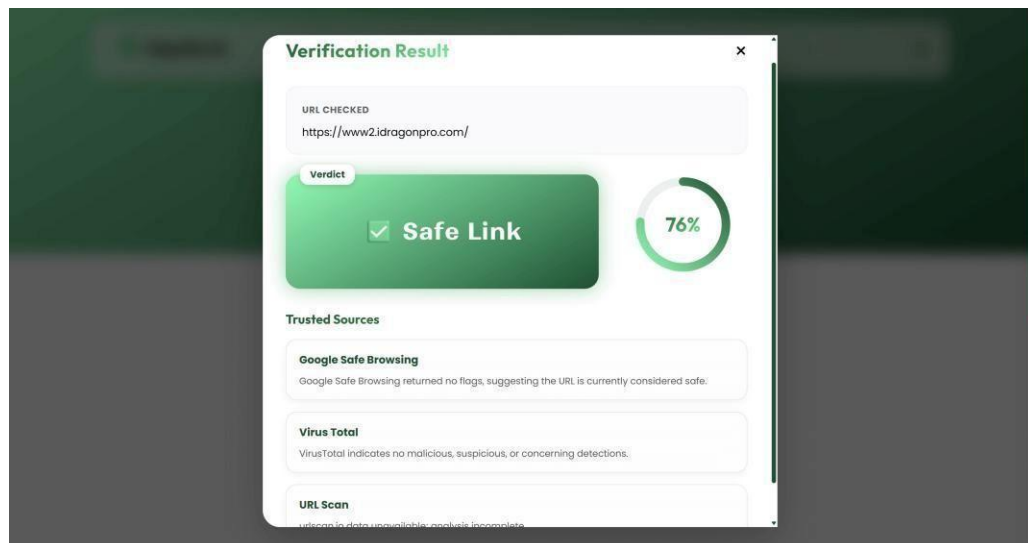
10.3. News Detecting Page



10.4. News Result Page



10.5. Link Detecting Page



10.6. Link Result Page

11. FUTURE ENHANCEMENTS

As technology evolves, fake images and deepfakes are becoming more realistic and harder to identify. In the future, Neethi.AI can be improved in several ways to make it more powerful and accurate in detecting such manipulations:

1. Smarter Deepfake Identification

Upcoming versions can use advanced AI models that analyze even the smallest visual clues like lighting, eye movement, and face texture to detect deepfakes more precisely. Combining multiple AI techniques will help recognize fake videos or photos faster and with higher accuracy.

2. Image Origin Tracking with Blockchain

Every verified image can be linked to a secure digital signature stored on a blockchain. This would make it easier to trace where the image came from and confirm whether it has been edited or reused elsewhere.

3. Detecting Identity Tampering

The system can be enhanced to recognize when a person's face has been digitally replaced or altered using face-mapping or 3D recognition. This feature would be especially useful in catching fake celebrity or political deepfakes.

4. Reverse Image Search Integration

Adding reverse image search capabilities can help the system automatically find where an image first appeared online and whether it has been posted before in a different context.

5. Real-Time Deepfake Scanning on Social Media

Future updates could include tools that monitor and analyze videos or images being shared on social media. This would help detect fake or harmful content before it spreads widely.

6. Support for Cybercrime and Media Verification

In the long term, Neethi.AI could work with law enforcement and media organizations to provide verified reports and help in tracking and preventing fake media incidents.

12. CONCLUSION

In conclusion, the Fake News and Malicious Content Detection Platform represent a significant step forward in the fight against digital misinformation and cyber threats. By combining advanced AI, real-time data verification, and user-centric design, the platform empowers users to make informed decisions and navigate the digital world with confidence.

The system's scalable, secure, and privacy-compliant architecture ensures long-term viability and trust. Its modular design and commitment to continuous improvement position it to adapt to evolving challenges and user needs. As the digital landscape continues to change, the platform's focus on ethical innovation, accessibility, and community engagement will help foster a safer, more informed, and resilient online environment for all.

13. REFERENCES

Books:

1. HTML - Head First HTML
2. CSS - CSS: Design and Build Websites
3. JavaScript - Head First JavaScript Programming

Links:

1. Node js - <https://nodejs.org/docs/latest/api/>
2. n8n - <https://docs.n8n.io/>
3. Gemini AI Docs - <https://ai.google.dev/gemini-api/docs>